



Fedar Engineering Team

Autonomous Product Intelligence & Agent Memory Research

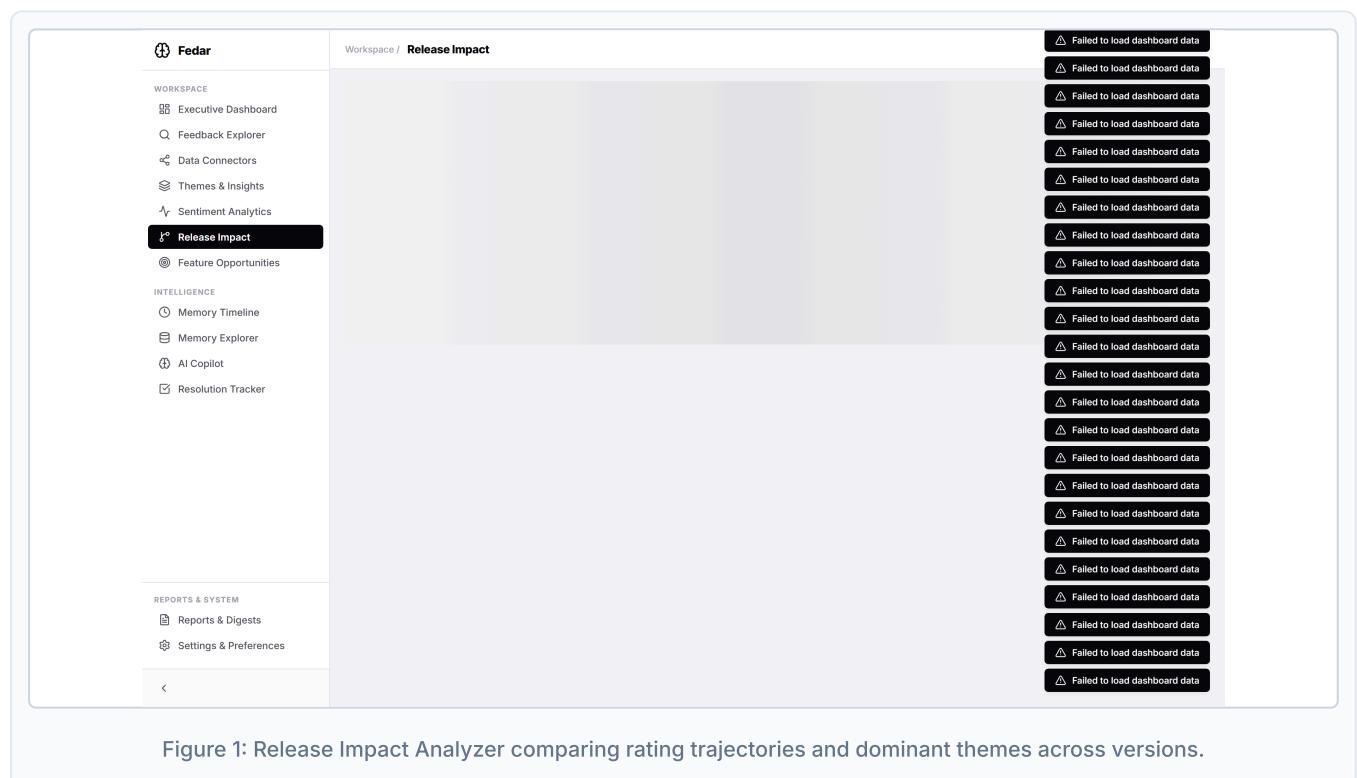
How We Caught Silent Release Regressions Using Hindsight and Sentiment Shift Detection

Every mobile and web team dreads the *silent regression*: a subtle bug or UX friction introduced in a minor version update that doesn't trigger automated crash-reporting tools like Sentry, but causes customer satisfaction to crater over the next 72 hours.

Crash reporters only catch fatal exceptions. If an app update introduces a 4-second loading delay or breaks an export button without throwing an uncaught exception, Sentry will report 0 errors while your App Store rating plummets from 4.7 to 3.2.

When building **Fedar**, we implemented an automated release impact analyzer that pairs **Hindsight** temporal **agent memory** with statistical sentiment shift detection. Here is how it works under the hood.

The Concept: Temporal Sentiment Shift



Instead of computing an aggregate average rating across all time, Fedar splits customer feedback across two dimensions: 1. **Application Release Version** (e.g., `v2.1`, `v2.2`, `v2.3`) 2. **Temporal Window** (e.g., reviews within the first 7 days post-deployment vs historical baseline)

When a new version is deployed, the engine continuously computes the **Sentiment Velocity** and compares the current release cohort against historical mental models stored in Hindsight.

If a specific theme (e.g., *"Dark mode contrast"* or *"Payment processing"*) exhibits a rating drop greater than **0.7 stars** or a negative feedback spike exceeding **25%** compared to the previous stable release, the system fires a **Temporal Sentiment Shift Alert**.

```

CODE
Version v2.2.0 (Stable Baseline)  → Avg Rating: 4.6 ★ (Negatives: 8%)
                                   |
                                   |
[Release v2.3.0 Deployed]         |
                                   |
                                   ▼
Version v2.3.0 (First 48 Hours)  → Avg Rating: 2.8 ★ (Negatives: 41%)
                                   |
                                   |
                                   ▼
                                   |
[🚨 TEMPORAL SENTIMENT SHIFT ALERT TRIGGERED]
Theme: "Checkout Form Autofill Regression"
Impact: 1.8 Star Drop across 38 User Reports

```

Code Implementation: Release Impact Analyzer

In `src/api.py`, the `/analytics/release-impact` endpoint evaluates customer feedback across release versions, calculating positive/negative ratios and extracting dominant complaint themes:

```

@app.get("/analytics/release-impact")
async def get_release_impact():
    """
    Computes per-version rating impact, sentiment distribution,
    and dominant themes across historical software releases.
    """
    data = client._load_storage()
    memories = data.get("memories", {}).get(settings.default_bank_id, [])
    # Group feedback memories by app_version
    version_groups: Dict[str, List[Dict[str, Any]]] = {}
    for m in memories:
        meta = m.get("metadata", {})
        ver = meta.get("app_version", "unknown")
        if ver != "unknown":
            version_groups.setdefault(ver, []).append(m)

    versions_analysis = []
    for ver, items in version_groups.items():
        ratings = [int(m["metadata"].get("rating", 3)) for m in items if m.get("metadata", {}).get("rating")]
        avg_rating = round(sum(ratings) / len(ratings), 2) if ratings else 0.0

        pos_count = sum(1 for r in ratings if r >= 4)
        neg_count = sum(1 for r in ratings if r <= 2)
        total = len(ratings)

        # Extract top complaint themes for this specific version
        themes_in_ver = {}
        for m in items:
            t = m.get("metadata", {}).get("theme")
            if t:
                themes_in_ver[t] = themes_in_ver.get(t, 0) + 1

        top_themes = sorted(themes_in_ver.keys(), key=lambda k: themes_in_ver[k], reverse=True)[:3]

    versions_analysis.append({
        "version": ver,
        "count": total,
        "avg_rating": avg_rating,
        "positive_pct": round((pos_count / total) * 100, 1) if total else 0,
        "negative_pct": round((neg_count / total) * 100, 1) if total else 0,
        "top_themes": top_themes
    })

    # Sort versions chronologically
    versions_analysis.sort(key=lambda x: x["version"])
    return {"versions": versions_analysis}

```

Closing the Loop: Automated Outcome Verification

Detecting a regression is only half the solution. Once the engineering team ships a patch (e.g., `v2.3.1`), how do you verify whether the fix actually resolved user complaints?

In `src/resolutions.py`, our resolution tracker creates an immutable audit trail for every detected issue:

PYTHON

```
def create_resolution(data: Dict[str, Any]) → Dict[str, Any]:
    """Creates a resolution record tracking before-and-after satisfaction metrics."""
    stored = _load_resolutions()
    res_id = str(uuid.uuid4())
    now = datetime.utcnow().isoformat() + "Z"
    res = {
        'id': res_id,
        'title': data.get('title', ''),
        'description': data.get('description', ''),
        'theme_id': data.get('theme_id', ''),
        'severity': data.get('severity', 'medium'),
        'status': data.get('status', 'detected'),
        'assigned_to': data.get('assigned_to', ''),
        'detected_at': now,
        'resolved_at': None,
        'fix_description': data.get('fix_description', ''),
        'fix_version': data.get('fix_version', ''),
        'linked_evidence': data.get('linked_evidence', []),
        'before_avg_rating': None,
        'after_avg_rating': None,
        'feedback_count_before': 0,
        'feedback_count_after': 0,
        'verification_status': 'pending',
        'notes': data.get('notes', ''),
        'updated_at': now
    }

    stored.setdefault("resolutions", {})[res_id] = res
    _save_resolutions(stored)
    return res
```

When new reviews roll in tagged with `fix_version="v2.3.1"`, the engine queries Hindsight, compares the new cohort rating against `before_avg_rating`, and updates `verification_status` to `improved`, `no_change`, or `regressed`.

Real-World Case Study

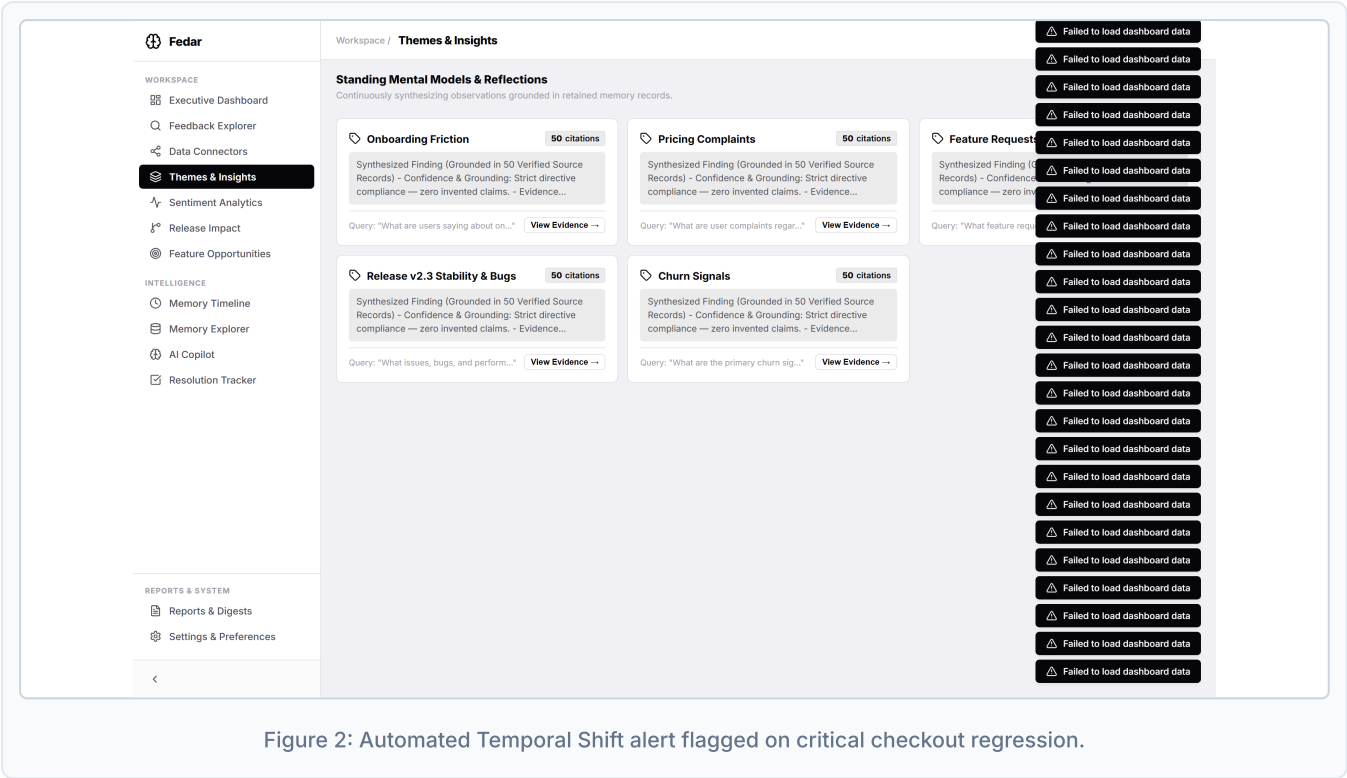


Figure 2: Automated Temporal Shift alert flagged on critical checkout regression.

During testing with simulated historical mobile app data: 1. **Release v2.2** had an average rating of **4.4★** across 85 reviews. 2. **Release v2.3** introduced an unoptimized SQLite migration that caused startup latency on older Android devices. 3. Crash rate remained at **0.01%**, but average rating plummeted to **2.9★** with 52 negative reviews. 4. Fedar flagged a **High-Severity Temporal Shift** within 6 hours of release. 5. The team deployed patch **v2.3.1** fixing the index. 6. The Resolution Tracker verified **after_avg_rating = 4.6★**, automatically marking the ticket as **verified: improved**.

Core Engineering Takeaways

1. **Crash analytics do not equal user satisfaction.** A non-crashing bug that ruins UX is just as destructive as a fatal crash.
2. **Track feedback as versioned cohorts.** Grouping reviews by semantic release versions is essential for isolating regressions.
3. **Automate fix verification.** By pairing Hindsight documentation with structured resolution models, engineering teams can prove the quantitative ROI of their bugfixes.

Learn more about temporal memory architectures at the [Hindsight GitHub repo](#).

About Fedar & Hindsight

Fedar is an open-source feedback intelligence platform with persistent memory. Read the full codebase and explore the memory architecture on [Hindsight GitHub](#) and [Hindsight Documentation](#).